

2026학년도 2학기 언어데이터과학

제2강 git 환경 만들기

박수민

서울대학교 인문대학 언어학과

2026년 9월 7일 월요일

지난 시간에 배운 것

- 1 강좌의 일정과 평가 방법
- 2 이 수업이 지향하는 “재현 가능한 연구”

오늘의 목표

- 1 버전 관리가 왜 필요한지 설명할 수 있다.
- 2 내 컴퓨터에 git 저장소를 만들고 commit할 수 있다.
- 3 GitHub 저장소에 push하고 pull할 수 있다.
- 4 GitHub Codespaces에 접속하여 실습 환경을 준비할 수 있다.

사전 준비 확인

- 1 VS Code(Visual Studio Code)가 설치되어 있다.
 - <https://code.visualstudio.com/download>
- 2 git이 설치되어 있다.
 - <https://git-scm.com/install/>
- 3 GitHub 계정이 있다. ([숙제01a])
 - <https://github.com>

아직 준비가 안 되었다면

지금 설치를 시작하고, 설치가 끝날 때까지 앞부분 설명을 먼저 듣는다.

1 버전 관리

왜 필요한가

git과 GitHub

2 실습: 로컬 저장소 만들기

3 실습: GitHub에 올리기

4 GitHub Codespaces

5 마무리

익숙한 풍경

과제 폴더의 어느 날

- 분석.py
- 분석_수정.py
- 분석_수정2.py
- 분석_최종.py
- 분석_진짜최종.py
- 분석_진짜최종_교수님피드백.py

생기는 문제

- 어느 파일이 최신인지 모른다.
- 무엇이 달라졌는지 모른다.
- 어제 잘 돌아가던 코드로 되돌아갈 수 없다.
- 논문의 그 표가 어느 코드에서 나왔는지 모른다.

버전 관리

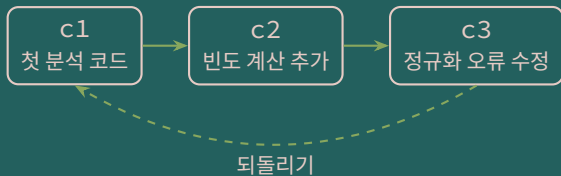
복사본 대신 이력

복사본 쌓기

- 분석.py
- 분석_최종.py
- 분석_진짜최종.py
- ...

이력 남기기

- 분석.py



버전 관리(version control)

새로운 파일을 복사하지 않고, **언제·누가·무엇을·왜 바꾸었는지**를 기록으로 남기는 방식.

언어학 연구와 버전 관리

재현 가능한 연구(reproducible research)

같은 데이터와 같은 코드로 같은 결과를 다시 얻을 수 있어야 한다.

버전 관리가 없으면

- “표 3의 빈도는 어떤 코드로 뽑았나요?” — 대답할 수 없다.
- “말뭉치를 다시 내려받았더니 수치가 달라졌어요.” — 무엇이 달라졌는지 알 수 없다.
- 6개월 뒤의 나는 남이다. 그 남에게 아무것도 설명해 줄 수 없다.

이 수업에서

한 학기 동안의 과제와 기말 프로젝트를 모두 **하나의 저장소**에 이력과 함께 남긴다.

git

정의

git은 파일의 변경 이력을 관리하는 분산 버전 관리 시스템(distributed version control system)이다.

특징

- 내 컴퓨터 안에 이력 전체가 들어 있다. (인터넷 없이도 기록 가능)
- 텍스트 파일(코드, 마크다운, CSV)의 변화를 줄 단위로 추적한다.
- 2005년 Linus Torvalds가 Linux 커널 개발을 위해 만들었다.

출처 <https://git-scm.com/about>

git과 GitHub는 다르다

	git	GitHub
정체	프로그램	웹 서비스
위치	내 컴퓨터	인터넷 서버
역할	이력을 기록한다	이력을 보관·공유한다
없으면	버전 관리를 할 수 없다	남과 공유할 수 없다
대체재	Mercurial, SVN	GitLab, Bitbucket

비유

git이 **카메라**라면, GitHub는 찍은 사진을 올려 두는 **사진첩 서버**이다.

핵심 용어

저장소(repository)

버전 관리의 단위가 되는 폴더. 폴더 안의 숨김 폴더 `.git`에 이력이 저장된다.

커밋(commit)

어느 시점의 파일 상태를 통째로 저장한 **기록 한 개**. 각 커밋은 다음을 가진다.

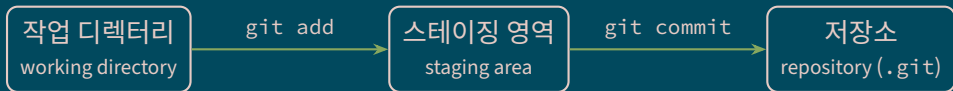
스냅샷 그 시점 파일들의 내용

메시지 무엇을 왜 바꾸었는지에 대한 설명

해시 `a1b2c3d...` 형태의 고유한 이름

저자·시각 누가 언제 만들었는지

git의 세 영역



읽는 법

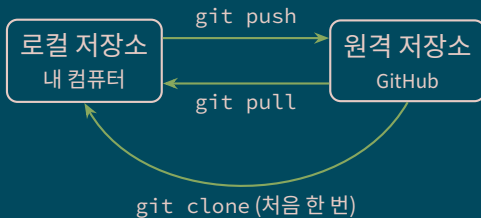
작업 디렉터리 내가 지금 편집하고 있는 파일들

스테이징 영역 이번 커밋에 포함시키기로 **고른** 변경사항

저장소 커밋으로 확정되어 이력에 남은 상태

파일을 저장(Ctrl+S)하는 것과 커밋하는 것은 다르다.

로컬 저장소와 원격 저장소



세 명령의 차이

clone 원격 저장소를 통째로 내 컴퓨터에 복사해 온다. (처음 한 번)

push 내가 만든 커밋을 원격 저장소로 올린다.

pull 원격 저장소의 새 커밋을 내 컴퓨터로 가져온다.

확인 질문

개념 확인

- 1 파일을 편집하고 저장만 했다. 이 변경사항은 세 영역 중 어디에 있는가?
- 2 `git add`만 하고 `git commit`을 하지 않으면 이력에 남는가?
- 3 집 컴퓨터에서 작업한 결과를 학교 컴퓨터에서 이어서 하려면 어떤 명령이 필요한가?

1 버전 관리

2 실습: 로컬 저장소 만들기

환경 설정
작업 폴더
첫 커밋

3 실습: GitHub에 올리기

4 GitHub Codespaces

5 마무리

지금부터 할 일

오늘의 실습 과정

- 1 git 설치와 계정 정보를 확인한다.
- 2 작업 폴더 LDS2026을 만들고 VS Code로 연다.
- 3 README.md 파일을 만든다.
- 4 폴더를 git 저장소로 만들고 첫 커밋을 남긴다.
- 5 GitHub에 저장소를 올린다.
- 6 변경 → 커밋 → 푸시를 한 번 더 반복한다.
- 7 GitHub Codespaces에 접속해 본다.

슬라이드 읽는 법

터미널 입력

이 상자에 있는 것은 터미널에 **그대로 입력**하는 명령이다.

실행 결과

이 상자에 있는 것은 명령을 실행하면 **나오는 출력**이다. 입력하지 않는다.

- 출력의 날짜·해시값·경로는 사람마다 다르다. 똑같지 않아도 정상이다.

주의

폴더 이름과 파일 이름은 **대소문자를 구별**한다.

git 설치 확인

터미널 입력

```
git --version
```

실행 결과

```
git version 2.51.0
```

이런 결과가 나오면

command not found 또는 인식할 수 없는 명령

→ git이 설치되지 않았거나, 설치 후 터미널을 다시 켜지 않은 것이다.

git에게 나를 소개하기

터미널 입력

```
git config --global user.name "your-username"  
git config --global user.email "your@e.mail"  
git config --global init.defaultBranch main
```

바꿔 넣을 것

your-username 자신의 GitHub username

your@e.mail GitHub에 가입할 때 쓴 이메일 주소

모든 커밋에는 저자 이름과 이메일이 함께 기록된다.

설정 확인하기

터미널 입력

```
git config --global --list
```

실행 결과

```
user.name=your-username  
user.email=your@e.mail  
init.defaultbranch=main
```

작업 폴더 만들기

파일 탐색기(Finder)에서 할 일

- 1 찾기 쉬운 곳(예: 문서 폴더)에 LDS2026 폴더를 만든다.
- 2 폴더 이름은 대문자 LDS + 2026. — 대소문자 주의!

피해야 할 위치

- 바탕화면처럼 파일이 뒤섞이는 곳
- 클라우드 동기화 폴더(iCloud Drive, OneDrive 등) — 동기화 충돌이 잦다.

VS Code에서 폴더 열기

VS Code에서 할 일

- 1 메뉴에서 [File] → [Open Folder]를 클릭한다.
- 2 방금 만든 LDS2026 폴더를 찾아서 연다.
- 3 신뢰 여부를 묻는 창이 뜨면 [Yes, I trust the authors]를 누른다.

첫 파일 만들기

VS Code에서 할 일

- 1 Activity Bar(왼쪽 세로 막대)에서 [Explorer] 배지를 클릭한다.
- 2 LDS2026 탭에서 [New File...] 아이콘을 클릭한다.
- 3 파일 이름으로 README.md를 입력한다.

README.md 작성하기

파일에 적을 내용

```
# LDS2026
```

- 학번: 2026-12345
- 이름: 김언어
- 이 저장소는 2026학년도 2학기 언어데이터과학 과제 저장소입니다.

README.md란

저장소를 처음 열었을 때 GitHub가 보여주는 안내문. 마크다운(Markdown) 형식으로 쓴다.

- 작성한 뒤 반드시 저장한다. (Ctrl+S / Cmd+S)

터미널 열기

VS Code에서 할 일

- 1 메뉴에서 [Terminal]을 클릭한다.
- 2 [New Terminal]을 클릭한다.
- 3 화면 아래에 터미널이 열린다.

확인

터미널의 현재 위치가 LDS2026 폴더인지 확인한다.

저장소 만들기

터미널 입력

```
git init -b main
```

실행 결과

```
Initialized empty Git repository in /Users/kim/Documents/LDS2026/.git/
```

일어난 일

LDS2026 폴더 안에 숨김 폴더 `.git`이 생겼다. 이제 이 폴더는 저장소이다.

- `-b main`: 첫 브랜치의 이름을 `main`으로 정한다.

상태 확인하기

터미널 입력

```
git status
```

실행 결과

```
On branch main
```

```
No commits yet
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
README.md
```

`git status`는 가장 자주 쓰는 명령이다. 막히면 일단 이것부터 실행한다.

커밋할 파일 고르기

터미널 입력

```
git add .  
git status
```

실행 결과

```
On branch main  
  
No commits yet  
  
Changes to be committed:  
  (use "git rm --cached <file>..." to unstage)  
    new file:   README.md
```

.은 현재 폴더 전체를 뜻한다. Untracked → to be committed로 바뀌었다.

첫 커밋 남기기

터미널 입력

```
git commit -m "학번과 이름을 적은 readme 추가"
```

실행 결과

```
[main (root-commit) a1b2c3d] 학번과 이름을 적은 readme 추가  
1 file changed, 5 insertions(+)  
create mode 100644 README.md
```

읽는 법

a1b2c3d가 이 커밋의 이름(해시)이다. 사람마다 다르다.

커밋 메시지 잘 쓰는 법

좋은 메시지

- “빈도 계산에서 대소문자 통합”
- “3강 실습 코드 추가”
- “표제어 추출 정규식 오류 수정”

나쁜 메시지

- “수정”
- “asdf”
- “최종”

기준

6개월 뒤의 내가 이 한 줄만 보고도 **무엇을 왜 바꾸었는지** 알 수 있어야 한다.

이력 확인하기

터미널 입력

```
git log --oneline
```

실행 결과

```
a1b2c3d (HEAD -> main) 학번과 이름을 적은 readme 추가
```

알아 둘 것

출력이 길어서 화면을 덮으면 q를 눌러 빠져나온다.

- 1 버전 관리
- 2 실습: 로컬 저장소 만들기
- 3 **실습: GitHub에 올리기**
저장소 발행
변경사항 주고받기
- 4 GitHub Codespaces
- 5 마무리

GitHub에 저장소 만들기

VS Code에서 할 일

- 1 Activity Bar에서 [Source Control] 배지를 클릭한다.
- 2 [Publish Branch] 버튼을 누른다.
- 3 GitHub에 로그인한다. (브라우저가 열린다)
- 4 [Publish to GitHub **private** repository]를 고른다.

왜 private인가

과제 코드가 공개되면 다른 수강생이 그대로 볼 수 있다. 교수자는 collaborator로 초대한다.

명령어로 하는 방법

터미널 입력

```
git remote add origin https://github.com/your-username/LDS2026.git  
git push -u origin main
```

읽는 법

origin 원격 저장소에 붙이는 관례적인 별명

-u 앞으로 git push만 쳐도 이 원격 저장소로 가도록 기억시킨다

GitHub에서 빈 저장소를 먼저 만든 뒤에 실행해야 한다.

올라간 것 확인하기

실행 결과

```
Enumerating objects: 3, done.  
Writing objects: 100% (3/3), 289 bytes | 289.00 KiB/s, done.  
To https://github.com/your-username/LDS2026.git  
* [new branch]      main -> main  
branch 'main' set up to track 'origin/main'.
```

웹 브라우저에서 할 일

<https://github.com>에 접속하여 LDS2026 저장소와 README.md의 내용을 확인한다.

변경사항 만들기

VS Code에서 할 일

- 1 LDS2026 폴더 안에 notes 폴더를 만든다.
- 2 notes/02-20260907.md 파일을 만든다.
- 3 오늘 수업에서 새로 알게 된 것을 세 줄로 적고 저장한다.

폴더 이름 규칙

회차 번호는 두 자리로, 날짜는 YYYYMMDD로 적는다. 이름이 규칙적이면 나중에 찾기 쉽다.

변경사항 올리기

터미널 입력

```
git add .  
git commit -m "2강 필기 추가"  
git push
```

앞으로 반복할 세 명령

- 1 add: 이번에 기록할 변경사항을 고른다.
- 2 commit: 내 컴퓨터의 이력에 남긴다.
- 3 push: GitHub로 올린다.

commit까지만 하면 GitHub에는 아무것도 올라가지 않는다.

웹에서 고치기

웹 브라우저에서 할 일

- 1 LDS2026 저장소에 들어간다.
- 2 `notes/02-20260907.md` 파일을 연다.
- 3 오른쪽 위 연필 아이콘을 눌러 내용을 고친다.
- 4 [Commit changes...] 버튼을 누른다.

이제 GitHub의 이력이 내 컴퓨터의 이력보다 **앞서 있다**.

변경사항 가져오기

터미널 입력

```
git pull
```

실행 결과

```
Updating a1b2c3d..e4f5a6b
Fast-forward
 notes/02-20260907.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

습관

작업을 시작하기 전에 `git pull`, 작업을 끝내면 `git push`.

다른 컴퓨터에서 이어서 하기

터미널 입력

```
git clone https://github.com/your-username/LDS2026.git
```

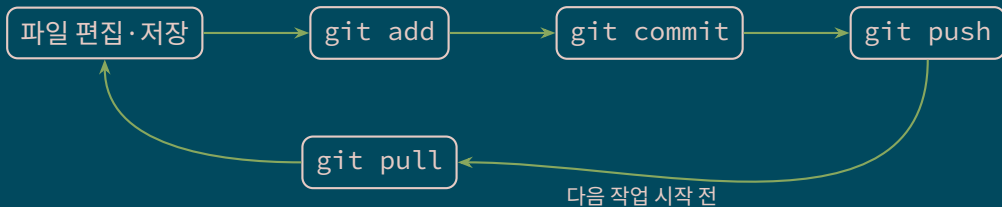
실행 결과

```
Cloning into 'LDS2026'...  
remote: Enumerating objects: 8, done.  
Receiving objects: 100% (8/8), done.
```

주의

clone은 저장소마다 컴퓨터마다 **처음 한 번만** 한다. 그 뒤로는 pull을 쓴다.

한 사이클 정리



저장소에 넣지 않는 것

- 수백 MB 이상의 말뭉치 원본 파일 (내려받는 방법을 대신 적는다)
- 개인정보가 담긴 파일, 비밀번호·API 키
- 이런 파일 이름은 .gitignore에 적어 두면 git이 무시한다.

확인 질문

실습 점검

- 1 내 저장소의 주소(URL)는 무엇인가?
- 2 지금까지 남긴 커밋은 몇 개인가? (`git log --oneline`으로 확인)
- 3 저장소가 private으로 만들어졌는가?

1 버전 관리

2 실습: 로컬 저장소 만들기

3 실습: GitHub에 올리기

4 **GitHub Codespaces**
실습 환경

5 마무리

왜 Codespaces를 쓰는가

문제

- 전산실 컴퓨터, 개인 노트북, 집 컴퓨터의 환경이 저마다 다르다.
- Windows에는 Linux 명령이 없고, 패키지 설치에서 막히면 수업이 멈춘다.

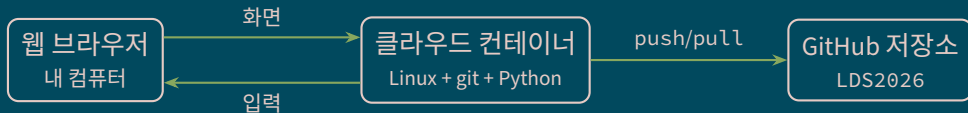
GitHub Codespaces

GitHub가 클라우드에 띄워 주는 **리눅스 개발 환경**. 웹 브라우저에서 VS Code와 터미널을 그대로 쓴다.

- 3강부터의 실습은 이 환경을 기준으로 진행한다.

출처 <https://docs.github.com/en/codespaces>

Codespaces의 구조



핵심

코드가 실행되는 곳은 내 컴퓨터가 아니라 **클라우드**이다. 내 컴퓨터에는 브라우저만 있으면 된다.

Codespaces 진입하기

웹 브라우저에서 할 일

- 1 LDS2026 저장소에 들어간다.
- 2 초록색 [Code] 버튼을 누른다.
- 3 [Codespaces] 탭을 고른다.
- 4 [Create codespace on main]을 누른다.

처음 만들 때는 1-2분 정도 걸린다.

Codespaces에서 확인하기

터미널 입력

```
pwd  
git status  
python3 --version
```

실행 결과

```
/workspaces/LDS2026  
On branch main  
Your branch is up to date with 'origin/main'.  
Python 3.12.1
```

저장소가 이미 clone된 상태로 열린다. git clone을 다시 할 필요가 없다.

Codespaces 사용 시 주의

반드시 기억할 것

- 1 Codespace 안에서 편집한 파일도 commit, push해야 저장소에 남는다.
- 2 다 쓰면 [Stop codespace]로 중지한다. 커 둔 시간만큼 사용량이 차감된다.
- 3 오래 쓰지 않으면 자동으로 삭제될 수 있다. 커밋하지 않은 파일은 함께 사라진다.

무료 사용량 (개인 계정 기준)

매월 120 core-hour(2코어 기준 60시간)와 15 GB 저장 공간이 무료로 제공된다.

출처 <https://docs.github.com/en/billing/concepts/product-billing/github-codespaces>

1 버전 관리

2 실습: 로컬 저장소 만들기

3 실습: GitHub에 올리기

4 GitHub Codespaces

5 **마무리**

요약

다음 시간 예고

과제

오늘 쓴 명령

명령	하는 일
<code>git init -b main</code>	현재 폴더를 저장소로 만든다
<code>git config</code>	이름·이메일 등 설정을 지정한다
<code>git status</code>	지금 어떤 상태인지 확인한다
<code>git add .</code>	변경사항을 이번 커밋에 포함시킨다
<code>git commit -m "..."</code>	이력에 기록을 남긴다
<code>git log --oneline</code>	지금까지의 이력을 본다
<code>git push</code>	커밋을 GitHub로 올린다
<code>git pull</code>	GitHub의 커밋을 가져온다
<code>git clone</code>	원격 저장소를 처음 복사해 온다

자주 만나는 오류

메시지	대처
command not found: git	git 설치 후 터미널을 다시 연다
Author identity unknown	git config로 이름·이메일을 설정한다
nothing to commit	파일을 저장했는지, add했는지 확인한다
Updates were rejected	git pull을 먼저 실행한 뒤 push한다
Permission denied	GitHub 로그인 계정이 맞는지 확인한다

막혔을 때

오류 메시지를 **끝까지 읽는다**. git은 다음에 할 일을 대개 함께 알려 준다.

확인 질문

정리

- 1 commit과 push의 차이는 무엇인가?
- 2 저장소를 private으로 만들었는데 교수자가 과제를 보려면 무엇이 필요한가?
- 3 Codespace를 끄지 않고 그대로 두면 어떤 일이 생기는가?

오늘 배운 것

- 1 버전 관리의 개념과 git의 세 영역
- 2 로컬 저장소 만들기: `init` → `add` → `commit`
- 3 GitHub와 주고받기: `push`, `pull`, `clone`
- 4 GitHub Codespaces 실습 환경

다음 시간에 배울 것 (9월 9일 수요일)

- 1 Linux 기초 명령어를 활용한 텍스트 파일 탐색

다음 시간까지 할 일

- 1 자기 저장소의 Codespace에 접속해 보기

[숙제01b] GitHub 저장소 만들기

- 1 LDS2026 저장소(private)를 만들고 README.md를 push한다.
- 2 저장소의 collaborator로 suparklingmin(교수자), gitMyeo(조교)을 초대한다.
 - 저장소 → [Settings] → [Collaborators] → [Add people]
- 3 eTL에 저장소 주소를 제출한다.
 - 제출 기한: 2026-09-08 09:00 (화요일 오전 9시)

잘 되었는지 확인하는 법

브라우저의 시크릿 창에서 저장소 주소를 열었을 때 404가 나오면 private이 맞다.